

is shown in Figure 2.

- **Managing global tools in Jenkins for JDK and Maven:** Before we start with pipeline we need to configure the Maven installation and JDK installation in the global tool configuration of Jenkins.
 1. To do so, go to *Manage Jenkins > Global tool configuration > Maven* and add the path to the Maven installation on the machine (MAVEN_HOME).
 2. Similarly, configure JAVA_HOME by going to *Manage Jenkins > Global tool configuration > JDK*.

Creating a Jenkins pipeline

Creating the initial pipeline in Jenkins:

1. Go to the Jenkins dashboard and click on *Open BlueOcean*.
2. Click on *Create a new Pipeline*.
3. Select *GitHub* from the available repository option in the BlueOcean dashboard.
4. Click on *Create an access token here* link available in the BlueOcean dashboard. You will be taken to the GitHub page to create *PAT*. Provide a name to an access token, copy the new access token, and save it in a safe place in case you want to refer to it later.
5. Paste the access token copied from GitHub into the BlueOcean dashboard, where we are trying to create a pipeline, and try to connect.
6. On successful connection, you will find all your organisations in the drop-down menu. Select the organisation from GitHub that contains the repositories that you want to use in the Blue Ocean pipeline.
7. Select a repository and click on *Create pipeline*.
8. Search for the pipeline in the Blue Ocean dashboard, and click on it. It will ask you again if you have a *Jenkinsfile* in the repository. So hit the *Create pipeline* option to create the Jenkins pipeline file in your project.
9. Give the name of the build agent as *Master* (or the name of the agent where you want to build the pipeline).
10. For now, you can add one stage 'Continuous Integration' with a sample echo bash script. Save and commit it to GitHub again.

Now we have the *Jenkinsfile* with the 'Continuous Integration' stage in our GitHub repository. We will just update this file in the next steps.

Unit test, Maven Java code and publishing the code coverage report in Jenkins: SonarQube is not capable of generating the units and code coverage results by itself. It imports the reports executed by the test framework used in the project. As we will refer to the Maven application, the test framework used in this implementation is Junit. And we will use Cobertura and Jacoco plugins from Maven for the generation of code coverage.

To do so we need to include these plugins in *pom.xml*.

The code below is the main file for all Maven project related configurations. Please refer to the *pom.xml* code below for unit test and code coverage of Java applications.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.7.5.201505241946</version>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>prepare-package</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
        <execution>
          <id>jacoco-check</id>
          <goals>
            <goal>check</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>cobertura-maven-plugin</artifactId>
      <version>${cobertura.version}</version>
      <configuration>
        <formats>
          <format>xml</format>
        </formats>
      </configuration>
      <executions>
        <execution>
          <phase>package</phase>
          <goals>
            <goal>cobertura</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    <plugin>
      <groupId>org.sonarsource.scanner.maven</groupId>
      <artifactId>sonar-maven-plugin</artifactId>
```